1.Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:
Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

Code:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int *arr = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    int low = 1, high = n;
    while (low < high) {
        int mid = low + (high - low) / 2;
        int count = 0;
```

```
        for (int i = 0; i < n; i++) {
            if (arr[i] <= mid) {
                count++;
            }
        }
        if (count > mid) {
            high = mid;
        } else {
            low = mid + 1;
        }
    }
    printf("%d\n", low);
    free(arr);
    return 0;
}
```

Output:

Input	Expected	Got
11 10 9 7 6 5 1 2 3 8 4 7	7	7
5 1 2 3 4 4	4	4
5 1 1 2 3 4	1	1

2. Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:
Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

Code:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int *arr = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    int tortoise = arr[0];
    int hare = arr[0];
    do {
        tortoise = arr[tortoise];
        hare = arr[arr[hare]];
    } while (tortoise != hare);
    tortoise = arr[0];
    while (tortoise != hare) {
        tortoise = arr[tortoise];
```

```
        hare = arr[hare];
    }
    printf("%d\n", tortoise);
    free(arr);
    return 0;
}
```

Output:

Input	Expected	Got
11 10 9 7 6 5 1 2 3 8 4 7	7	7
5 1 2 3 4 4	4	4
5 1 1 2 3 4	1	1

3. Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array

2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

Code:

```
#include <stdio.h>
#include <stdlib.h>

void findIntersection() {
    int n1, n2;
    if (scanf("%d", &n1) != 1) return;
    int *arr1 = (int *)malloc(n1 * sizeof(int));
    for (int i = 0; i < n1; i++) {
        scanf("%d", &arr1[i]);
    }
```

```
if (scanf("%d", &n2) != 1) {
    free(arr1);
    return;
}
int *arr2 = (int *)malloc(n2 * sizeof(int));
for (int i = 0; i < n2; i++) {
    scanf("%d", &arr2[i]);
}
int i = 0, j = 0;
int first = 1;
while (i < n1 && j < n2) {
    if (arr1[i] < arr2[j]) {
        i++;
    } else if (arr1[i] > arr2[j]) {
        j++;
    } else {
        if (!first) {
            printf(" ");
        }
        printf("%d", arr1[i]);
        first = 0;
        i++;
        j++;
    }
}
printf("\n");
free(arr1);
free(arr2);
```

```
}


int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        findIntersection();
    }
    return 0;
}
```

Output:

Input	Expected	Got
1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57
1 6 1 2 3 4 5 6 2 1 6	1 6	1 6

4.Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

Code:

```
#include <stdio.h>

#include <stdlib.h>
```

```
void findIntersection() {
    int n1, n2;
    if (scanf("%d", &n1) != 1) return;
    int *arr1 = (int *)malloc(n1 * sizeof(int));
    for (int i = 0; i < n1; i++) {
        scanf("%d", &arr1[i]);
    }
    if (scanf("%d", &n2) != 1) {
        free(arr1);
        return;
    }
    int *arr2 = (int *)malloc(n2 * sizeof(int));
    for (int i = 0; i < n2; i++) {
        scanf("%d", &arr2[i]);
    }
    int i = 0, j = 0;
    int first = 1;
    while (i < n1 && j < n2) {
        if (arr1[i] < arr2[j]) {
            i++;
        } else if (arr1[i] > arr2[j]) {
            j++;
        } else {
            if (!first) {
                printf(" ");
            }
            printf("%d", arr1[i]);
            first = 0;
```

```
            i++;
            j++;
        }
    }
    printf("\n");
    free(arr1);
    free(arr2);
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        findIntersection();
    }
    return 0;
}
```

Output:

Input	Expected	Got
1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57
1 6 1 2 3 4 5 6 2 1 6	1 6	1 6

5-Pair with Difference-O(n^2)Time Complexity,O(1) Space Complexity

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:
1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

Code:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int *arr = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
```

```
    }
    int k;
    if (scanf("%d", &k) != 1) {
        free(arr);
        return 0;
    }
    int i = 0, j = 1;
    int found = 0;
    while (i < n && j < n) {
        if (i != j && arr[j] - arr[i] == k) {
            found = 1;
            break;
        } else if (arr[j] - arr[i] < k) {
            j++;
        } else {
            i++;
        }
    }
    printf("%d\n", found);
    free(arr);
    return 0;
}
```

Output:

Input	Expected	Got
3 1 3 5 4	1	1
10 1 4 6 8 12 14 15 20 21 25 1	1	1
10 1 2 3 5 11 14 16 24 28 29 0	0	0
10 0 2 3 7 13 14 15 20 24 25 10	1	1

6.Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:
1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

Code:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int *arr = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    int k;
    if (scanf("%d", &k) != 1) {
        free(arr);
        return 0;
    }
    int i = 0, j = 1;
    int found = 0;
    while (i < n && j < n) {
        if (i != j && arr[j] - arr[i] == k) {
            found = 1;
            break;
        } else if (arr[j] - arr[i] < k) {
```

```
            j++;
        } else {
            i++;
        }
    }
    printf("%d\n", found);
    free(arr);
    return 0;
}
```

Output:

Input	Expected	Got
3 1 3 5 4	1	1
10 1 4 6 8 12 14 15 20 21 25 1	1	1
10 1 2 3 5 11 14 16 24 28 29 0	0	0
10 0 2 3 7 13 14 15 20 24 25 10	1	1